Experiment Title: Solving 2 water jug Problem

Aim : Write a program to solve the 2-water jug problem by manually providing moves from the user.

Theory: The Two Water Jug problem is used to understand basic logical thinking. In this problem, two jugs of fixed capacity are given and we have to get a required amount of water. We can only fill a jug, empty a jug, or pour water from one jug to another. By performing these steps properly, the target amount of water can be achieved.

Algorithm / Methodology:

We have total six operations

Fill X

Fill Y

Empty X

Empty Y

PourYtoX

PourXtoY

START

Initialize X = 0, Y = 0

Set capacities of X and Y

Set target value

WHILE target is not reached

    Display available operations

    Read user choice(switch case)

    Perform selected operation

    Update values of X and Y

    Display current state

END WHILE

STOP

Dataset Description:

Source: User Defined Input

Attributes:

X Jug Capacity=3

Y Jug Capacity=5

Target = [0,4]

Implementation:

```
def fillX(x,xCap):
  return xCap
def fillY(y,yCap):
  return yCap
def emptyX(x):
  x=0
  return 0
def emptyY(y):
  y=0
  return 0
def pourXtoY(x,y,yCap):
  spacey=yCap-y
  pour=min(x,spacey)
  x=x-pour
  y=y+pour
```

```
  return x,y
def pourYtoX(x,y,xCap):
  spacex=xCap-x
  pour=min(y,spacex)
  x=x+pour
  y=y-pour
  return x,y
x=0
y=0
xCap=3
yCap=5
Target=[0,4]
def twoWaterJug(xCap,yCap,Target):
  x,y=0,0
  xCap=3
  yCap=5
  Target=[0,4]
  while([x,y]!=Target):
    ch=int(input("Enter your choice:\n1.fill x\n2.fill y\n3.empty
x\n4.empty y\n5.pour x to y\n6.pour y to x\n7.exit\n"))
    match ch:
      case 1:
        x=fillX(x,xCap)
        print("x=",x)
      case 2:
        y=fillY(y,yCap)
        print("y=",y)
      case 3:
```

```
        x=emptyX(x)
        print("x=",x)
      case 4:
        y=emptyY(y)
        print("y=",y)
      case 5:
        x,y=pourXtoY(x,y,yCap) #pour x to y
        print("x=",x)
        print("y=",y)
      case 6:
        x,y=pourYtoX(x,y,xCap) #pour y to x
        print("x=",x)
        print("y=",y)
      case 7:
        break

    if(x==0 and y==4):
      print("Target reached")
      break
  return x,y

x,y=(twoWaterJug(xCap,yCap,Target))
```

Output / Result:

```
Enter your choice:
1.fill x
2.fill y
3.empty x
4.empty y
5.pour x to y
6.pour y to x
7.exit
2
y= 5
Enter your choice:
1.fill x
2.fill y
3.empty x
4.empty y
5.pour x to y
6.pour y to x
7.exit
6
x= 3
y= 2
Enter your choice:
1.fill x
2.fill y
3.empty x
4.empty y
5.pour x to y
6.pour y to x
7.exit
3
x= 0
```

```
Enter your choice:
1.fill x
2.fill y
3.empty x
4.empty y
5.pour x to y
6.pour y to x
7.exit
6
x= 2
y= 0
Enter your choice:
1.fill x
2.fill y
3.empty x
4.empty y
5.pour x to y
6.pour y to x
7.exit
2
y= 5
Enter your choice:
1.fill x
2.fill y
3.empty x
4.empty y
5.pour x to y
6.pour y to x
7.exit
6
x= 3
y= 4
Enter your choice:
1.fill x
2.fill y
3.empty x
4.empty y
5.pour x to y
6.pour y to x
7.exit
3
x= 0
Target reached
```

Result Analysis:

The output shows that the required amount of water is successfully obtained by performing valid operations on the jugs. All steps follow the given rules and do not exceed the jug capacities. Total 7 operations are performed if we fill y first.

Conclusion:

From this experiment, we learned how to solve the Two Water Jug problem using logical steps. The experiment helped in understanding state changes, valid operations, and goal checking. It also improved our problem-solving skills and understanding of basic concepts used in artificial intelligence.

Experiment Title: Missionaries and Cannibals problem

Aim: Write a program to implement Missionaries and Cannibals problem.
Constraints: three missionaries and three cannibals must cross a river using a boat which can carry at most two people, under the constraint that, for both banks, if there are missionaries present on the bank, they cannot be outnumbered by cannibals (if they were, the cannibals would eat the missionaries). The boat cannot cross the river by itself with no people on board.

Theory:

The Missionaries and Cannibals problem is a classical problem used to understand state space search in artificial intelligence. In this problem, three missionaries and three cannibals are on left side of a river along with a boat on the left bank. The goal is to move all of them to the right side.

The main condition of the problem is that cannibals should never be greater than missionaries on either side of the river otherwise they will eat the missionaries, unless there are no missionaries present. Each situation is represented as a state, and moving people using the boat changes the state. By checking valid states and avoiding unsafe ones, the problem can be solved systematically using BFS.

Algorithm / Methodology:

START

Initialize number of missionaries and cannibals on left bank

Set boat position to left

Define initial state and goal state

Create an empty queue and visited list

Add initial state to queue

WHILE queue is not empty

  Remove a state from the queue

  IF state is goal state

```
        Display solution path
        STOP
    END IF

    Generate all possible next states
    FOR each next state
        IF state is valid and not visited
            Add state to queue
            Mark state as visited
        END IF
    END FOR
END WHILE
STOP
```

Dataset Description:

Total Missionaries=3

Total Cannables=3

Initial=(3,3,0)

Final=(0,0,1)

Implementation:

```
t_miss=3
t_can=3

def constraints(m,c):
```

```
  if(m<0 or m>t_miss or c<0 or c>t_can):
    return False

  op_m=t_miss-m
  op_c=t_can-c
  # miss>=can
  if(m<c and m>0):
    return False
  if(op_m<op_c and op_m>0):
    return False
  return True

def next_state(pos):
  m,c,b=pos
  valid_state=[(0,1),(1,0),(1,1),(2,0),(0,2)]
  next=[]
  for i in valid_state:
    if(b==0):
      # left to right
      next_m=m-i[0]
      next_c=c-i[1]
      if constraints(next_m,next_c):
        next.append((next_m,next_c,1))
    else:
      # right to lft
      next_m=m+i[0]
      next_c=c+i[1]
      if constraints(next_m,next_c):
```

```
        next.append((next_m,next_c,0))
  return next


def miss_can():
  initial=(3,3,0)
  final=(0,0,1)
  queue=[(initial,[initial])]
  visited={initial}
  while queue:
    pos,path=queue.pop(0)
    if(pos==final):
      return path
    for i in next_state(pos):
      if i not in visited:
        queue.append((i,path+[i]))
        visited.add(i)
  return;


print("****** MISSIONARIES AND CANNIBALS: ******\n")
miss_can()
```

Output / Result:

```
****** MISSIONARIES AND CANNIBALS: ******

[(3, 3, 0),
 (2, 2, 1),
 (3, 2, 0),
 (3, 0, 1),
 (3, 1, 0),
 (1, 1, 1),
 (2, 2, 0),
 (0, 2, 1),
 (0, 3, 0),
 (0, 1, 1),
 (0, 2, 0),
 (0, 0, 1)]
```

Result Analysis:

So, We get the output which shows that BFS explores states level by level and avoids unsafe states using constraints defined. The solution obtained is optimal and reaches the goal state safely. Total 12 operations are required for reaching the goal safely.

Conclusion:

The experiment showed how to safely move missionaries and cannibals across the river using the given constraints. BFS helps find us the correct sequence of moves while avoiding unsafe states.

Viva Questions:

Thought Provoking question

1. Why is BFS preffered over DFS here?
   Because BFS arrives at the solution in less number of steps than DFS.
   DFS might take longer time to find correct path.

2. What happens if boat capacity is increased to 3?
   Then the problem will become more easier as now fewer steps will be required.

3. Can the water jug problem be solved using BFS instead of manual input?
   Yes, without user input also we can solve this problem using bfs.

4. How do constraints prune invalid states?
   Constrains are defined according to invalid conditions so while executing the unsafe conditions automatically gets prune.